



Recurrent Neural Networks

Computation and Discrete Structures III – Unit 3: Turing Machines and Applications

Juan Marcos Caicedo Mejía – jmcaicedo@icesi.edu.co

Department of Computing and Intelligent Systems
Barberi Faculty of Engineering, Design and Applied Sciences
ICESI University

Material adapted from Professors Andres Aristizabal, Angela Villota

1. Introduction to Recurrent Neural Networks

1.1 Exercises

1. Introduction to Recurrent Neural Networks

1.1 Exercises

Generally speaking, what is a recurrent neural network?

Generally speaking, what is a recurrent neural network?

- It is a type of artificial neural network commonly used in speech recognition and natural language processing.
- Recurrent neural networks recognize data's sequential characteristics and use patterns to predict the next likely scenario.
- They are especially powerful in use cases where context is critical to predicting an outcome.
- They are distinct from other types of artificial neural networks because they use feedback loops to process a sequence of data that informs the final output.
- These feedback loops allow information to persist.
- This effect often is described as memory.

What is the motivation for using such a neural network?

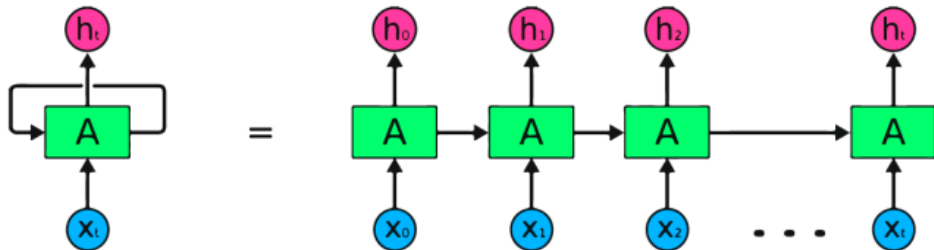
What is the motivation for using such a neural network?

- Not all problems can be converted into one with fixed length inputs and outputs.
- Problems such as speech recognition or time-series prediction require a system to store and use context.
- Hard, or even impossible to choose a fixed context window information.

What is the main difference between a Dense NN and a RNN?

What is the main difference between a Dense NN and a RNN?

- In the traditional neural network, the inputs and the outputs are independent of each other.
- The output in RNN is dependent on prior elements within the sequence.
- Recurrent networks also share parameters across each layer of the network.
- In feedforward networks, there are different weights across each node.
- Whereas RNN shares the same weights within each layer of the network and during gradient descent, the weights and basis are adjusted individually to reduce the loss.

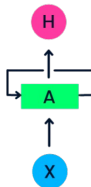


How do RNN work?

How do RNN work?

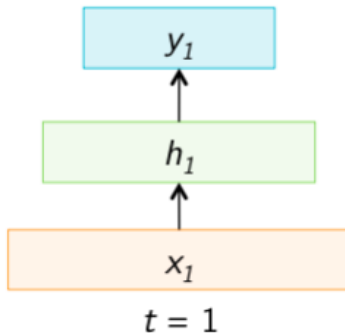
- They take the previous output or hidden states as inputs.
- The composite input at time t has some historical information about the happenings at time $T < t$
- Useful as their intermediate values (state) can store information about past inputs for a time that is not fixed *a priori*

How do RNN work?

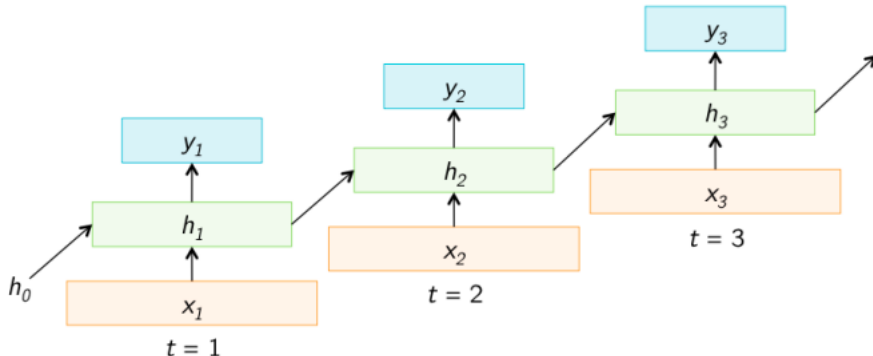


- The information cycles through the loop, so the output is determined by the current input and previous received inputs.
- The input layer X processes the initial input and passes it to the middle layer A .
- The middle layer consists of multiple hidden layers, each with its activation functions, weights, and biases.

Looking back at a feed forward network

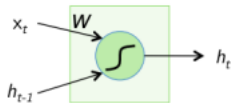


Let's see a RNN



How does a vanilla RNN cell work?

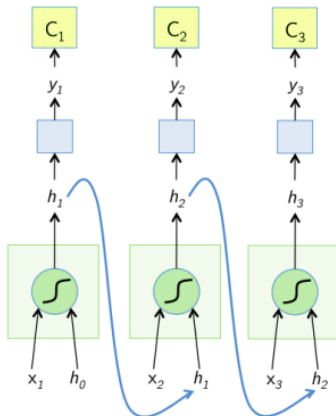
How does a vanilla RNN cell work?



$$h_t = \tanh W \begin{pmatrix} x_t \\ h_{t-1} \end{pmatrix}$$

And now the vanilla RNN?

And now the vanilla RNN?



$$h_t = \tanh W \begin{pmatrix} x_t \\ h_{t-1} \end{pmatrix}$$

$$y_t = F(h_t)$$

$$C_t = \text{Loss}(y_t, GT_t)$$

What else can we say about a RNN?

What else can we say about a RNN?

- The weights are shared over time
- Essentially, copies of the RNN cell are made over time (unrolling/unfolding), with different inputs at different time steps

How do we train our RNN?

How do we train our RNN?

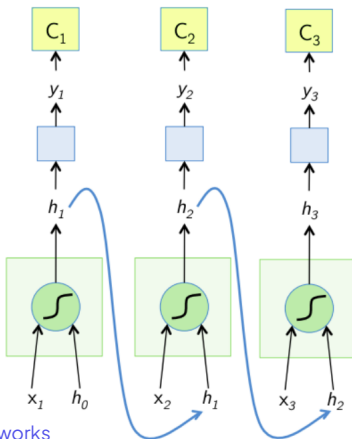
- We can use BackPropagation Through Time (BPTT)
- The unfolded network (used during forward pass) is treated as one big feed-forward network
- This unfolded network accepts the whole sequence as an input
- The weight updates are computed for each copy in the unfolded network, then summed (or averaged) and then applied to the RNN weights

How does BPTT work?

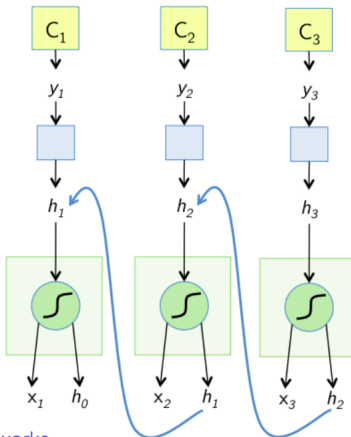
How does BPTT work?

- Treat the unfolded network as one big feed-forward network
- This big network takes in entire sequence as an input
- Compute gradients through the usual backpropagation
- Update shared weights

RNN in action



RNN in action

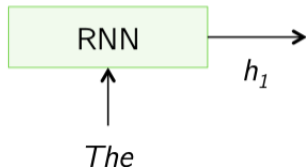


How about an example, how does a RNN work for sentiment classification?

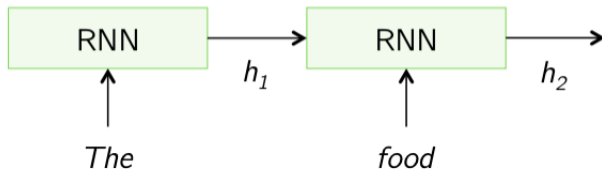
How about an example, how does a RNN work for sentiment classification?

- Classify a restaurant or movie review as positive or negative
- Inputs: Multiple words, one or more sentences
- Outputs: Positive / Negative classification

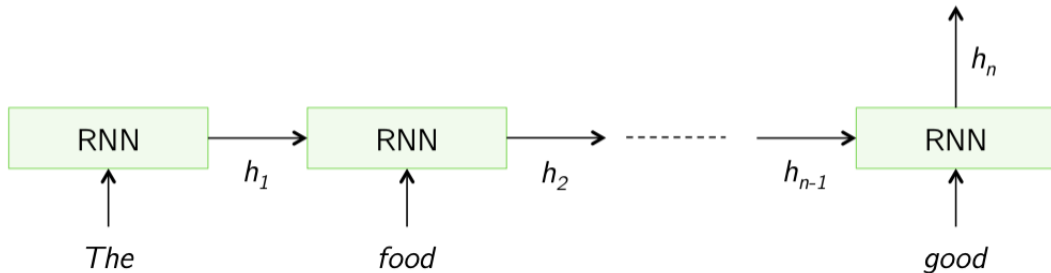
RNN in action



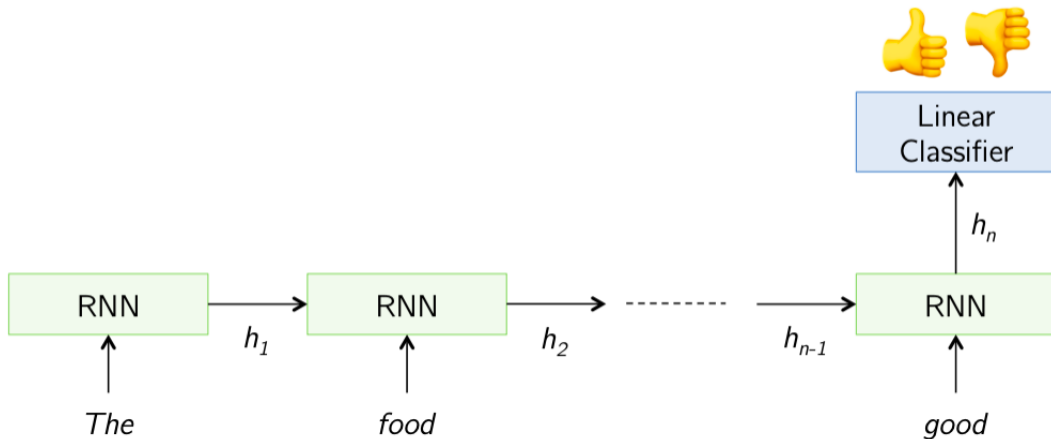
RNN in action



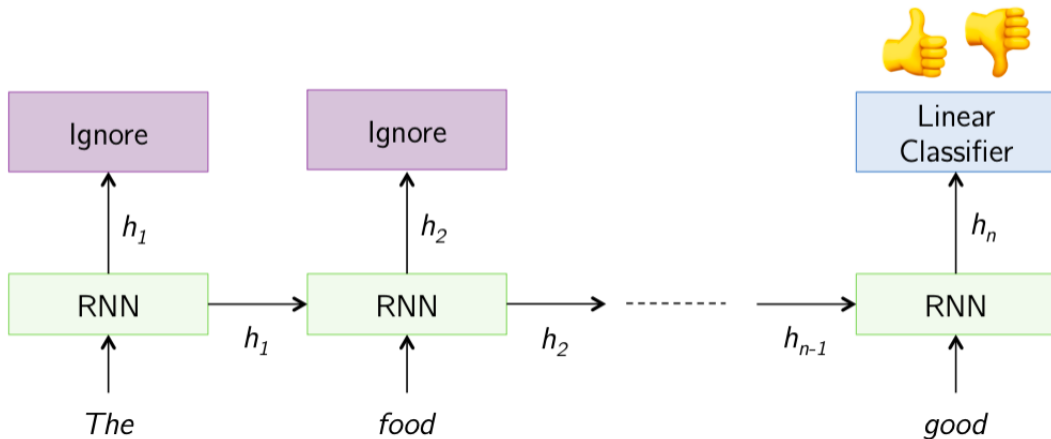
RNN in action



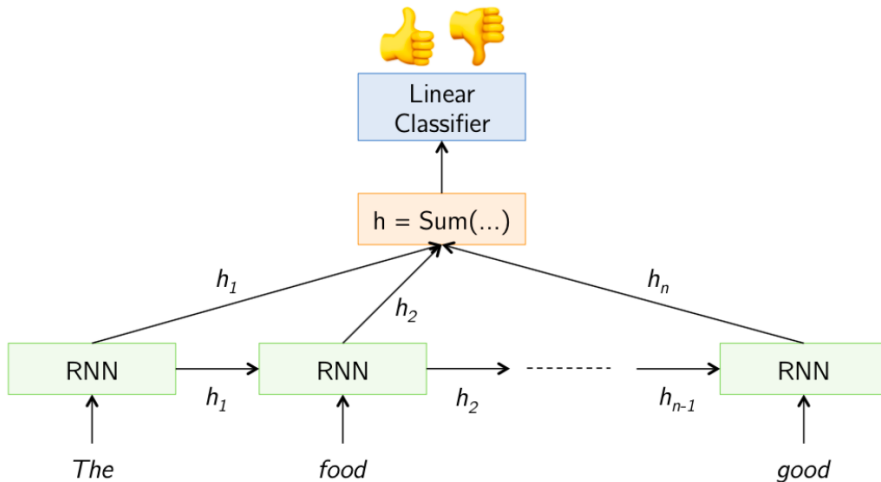
RNN in action



RNN in action



Another option?



How can we create a simple RNN using Keras and Tensorflow?

How can we create a simple RNN using Keras and Tensorflow?

- Import the right modules
- Add layers to the model
- Load the given dataset
- Compile the Recurrent Neural Network
- Train and fit the model
- Test the RNN model

Importing the necessary libraries

```
1 import tensorflow as tf
2 from tensorflow import keras
3 from tensorflow.keras import layers
```

Adding layers to the model

```
1 model = keras.Sequential()  
2 model.add(layers.SimpleRNN(64, input_shape=(None, 28)))  
3 model.add(layers.BatchNormalization())  
4 model.add(layers.Dense(10))  
5 print(model.summary())
```

Model: "sequential"

Layer (type)	Output Shape	Param #
simple_rnn (SimpleRNN)	(None, 64)	5,952
batch_normalization (BatchNormalization)	(None, 64)	256
dense (Dense)	(None, 10)	650

Total params: 6,858 (26.79 KB)

Trainable params: 6,730 (26.29 KB)

Non-trainable params: 128 (512.00 B)

None

Loading the MNIST dataset

```
1 mnist = keras.datasets.mnist
2 (x_train, y_train), (x_test, y_test) = mnist.load_data()
3 x_train, x_test = x_train/255.0, x_test/255.0
4 sample, sample_label = x_test[0], y_test[0]
```


Compiling the RNN

```
1 model.compile(loss=keras.losses.SparseCategoricalCrossentropy(  
2     from_logits=True),  
3     optimizer="sgd", metrics=["accuracy"],)
```

Training the RNN

```
1 model.fit(x_train, y_train, validation_data=(x_test, y_test),  
2         batch_size=64, epochs=10)
```

```
Epoch 1/10  
938/938 ————— 11s 10ms/step - accuracy: 0.7932 - loss: 0.6684 - val_accuracy: 0.8994 - val_loss: 0.3512  
Epoch 2/10  
938/938 ————— 9s 9ms/step - accuracy: 0.9068 - loss: 0.3221 - val_accuracy: 0.9023 - val_loss: 0.3184  
Epoch 3/10  
938/938 ————— 8s 8ms/step - accuracy: 0.9284 - loss: 0.2449 - val_accuracy: 0.9282 - val_loss: 0.2412  
Epoch 4/10  
938/938 ————— 9s 10ms/step - accuracy: 0.9388 - loss: 0.2092 - val_accuracy: 0.9457 - val_loss: 0.1843  
Epoch 5/10  
938/938 ————— 8s 9ms/step - accuracy: 0.9466 - loss: 0.1839 - val_accuracy: 0.9453 - val_loss: 0.1865  
Epoch 6/10  
938/938 ————— 8s 9ms/step - accuracy: 0.9513 - loss: 0.1671 - val_accuracy: 0.9532 - val_loss: 0.1620  
Epoch 7/10  
938/938 ————— 11s 9ms/step - accuracy: 0.9552 - loss: 0.1532 - val_accuracy: 0.9590 - val_loss: 0.1401  
Epoch 8/10  
938/938 ————— 9s 9ms/step - accuracy: 0.9577 - loss: 0.1435 - val_accuracy: 0.9569 - val_loss: 0.1470  
Epoch 9/10  
938/938 ————— 7s 8ms/step - accuracy: 0.9600 - loss: 0.1373 - val_accuracy: 0.9560 - val_loss: 0.1492  
Epoch 10/10  
938/938 ————— 9s 10ms/step - accuracy: 0.9623 - loss: 0.1275 - val_accuracy: 0.9640 - val_loss: 0.1168
```

Testing the RNN

```
1 result = tf.argmax(model.predict(tf.expand_dims(sample, 0)),  
2     axis=1)  
3 print(result.numpy(), sample_label)
```

1/1  0s 419ms/step
[7] 7

1. Introduction to Recurrent Neural Networks

1.1 Exercises

- Load the cifar10 dataset
- Train the a rnn model replicating the process explained in class taking into account certain tweaks.
- Use 20 epochs.

```
1  cifar10 = keras.datasets.cifar10.load_data()
2
3  (x_train, y_train), (x_test, y_test) = cifar10
4
5  x_train = x_train.astype('float32') / 255.0
6  x_test = x_test.astype('float32') / 255.0
7
8  y_train = y_train.ravel()
9  y_test = y_test.ravel()
10
11 x_train = x_train.reshape((x_train.shape[0], 32, 32 * 3))
12 x_test = x_test.reshape((x_test.shape[0], 32, 32 * 3))
13
14 model.add(layers.SimpleRNN(64, input_shape=(32, 32 * 3)))
```